Vinícius dos Santos Tomé - 232006780

- **Qual a razão do embaralhamento dos bits do imediato no RiscV?**

O embaralhamento dos bits entre os diferentes formatos de instrução tem como objetivo manter os campos essenciais sempre na mesma posição. A principal motivação é **manter o bit de sinal sempre localizado no bit 31 da instrução** para todos os formatos (I, S, SB, UJ). Isso simplifica drasticamente a arquitetura de hardware, pois o módulo de extensão de sinal pode começar a operar paralelamente, antes mesmo do processador terminar de decodificar qual é o formato exato da instrução. Além disso, reusar a posição de campos lógicos reduz o custo com multiplexadores no caminho de dados.

- **Por que alguns imediatos não incluem o bit 0?**

Os imediatos dos tipos SB (Branches) e UJ (Jumps) representam endereços/deslocamentos de memória. Como a arquitetura RISC-V exige que as instruções estejam alinhadas em memória (em múltiplos de 2 bytes com a extensão comprimida "C", ou múltiplos de 4 bytes sem ela), **os endereços de destino de um salto ou desvio sempre serão números pares**. Portanto, o bit 0 (bit menos significativo) do endereço sempre será 0. Não armazenar esse zero óbvio na instrução permite utilizar esse espaço para alcançar endereços duas vezes mais distantes, otimizando o tamanho do imediato (dobrando o alcance do salto).

- **Os imediatos de operações lógicas estendem o sinal?**

Sim. Diferente de algumas outras arquiteturas (como o MIPS antigo, que realizava extensão por zeros em imediatos lógicos), no RISC-V o dado imediato de 12 bits do tipo I-type *sempre* sofre extensão de sinal até 32 bits, independentemente da instrução ser aritmética (como addi) ou lógica (como andi, ori, xori). A única exceção parcial é o formato de shift (srai, slli, srli), onde apenas os 5 bits inferiores (shamt) são usados e os

bits superiores se tornam zero para representar a quantidade exata de deslocamento.

Como é implementada a instrução NOT no RiscV?

A arquitetura RISC-V adota uma filosofia de não incluir instruções redundantes se a mesma operação puder ser feita com instruções existentes. Portanto, a instrução NOT (negação lógica bit a bit) não existe nativamente. Ela é implementada como uma "pseudo-instrução" utilizando a instrução XORI (Exclusive OR Immediate) operando com o imediato -1.

O valor -1 em complemento de dois é preenchido com bits 1 (0xFFFFFFFF). Realizar a operação de Ou-Exclusivo entre um valor e "todos os bits em nível alto" inverte perfeitamente os bits da origem, obtendo o mesmo resultado da função NOT. (Sintaxe: xori rd, rs1, -1).